# Statistical equivalence of jmbayes and JMbayes2

Joint model for longitudinal and time-to-event data, ported from the R package JMbayes2 (v0.6-5) to Python/PyTorch. Same model, same priors and same MCMC algorithm; it runs on CPU or GPU. This report tests whether the two programs give the same answers on the same data. All numbers were measured on one machine: Intel Core i9-12900K (x86-64, 16 cores - 8 performance + 8 efficiency - and 24 threads), 64 GB RAM, NVIDIA GeForce RTX 3090 (24 GB); Windows 11, R 4.6.2 with JMbayes2 0.6-5, Python 3.11 with PyTorch 2.6 (CUDA 12.4).

• **R vs Python differs no more than R vs R.** Each program was run twice with different random seeds. Two R runs differ from each other by a median of 0.02 posterior SDs (largest 0.14); two Python runs by 0.05 (largest 0.20); R and Python by 0.02 (largest 0.16). The choice of implementation has no effect beyond the ordinary run-to-run noise of either program.

• **Same conclusions.** For all 15 of 15 regression coefficients (survival and longitudinal), R and Python agree on the direction of the effect, on significance at 5% and on whether the 95% credible interval excludes zero. Hazard ratios, intervals and p-values are shown side by side in Section 1.

• **Formal equivalence.** In an equivalence test (TOST, $\alpha = 0.05$) 45 of 50 parameters are proven equivalent within ±0.2 posterior SD (37 within ±0.1 SD). For the remaining 5 the runs are not precise enough to decide; no parameters are shown to differ. With realistic error bars, 98% of differences lie within 2 standard errors (expected ~95%) and 100% within 3.

• **Faster as cohorts grow.** At 3,120 patients the MCMC took 202 s in R and 24 s in Python on the GPU (8.6x). Python's time grows slowly with patients; R's grows roughly in proportion (Section 2).

• **Flat memory.** R's peak RAM grows with patients and with iterations (up to 6.2 GB at 1,560 patients x 28,000 iterations); Python stays near 1.3 GB, because it does not store a per-patient log-likelihood for every iteration.

• **Small data.** On the original 312 patients R and Python are comparable; most of R's time there is fixed start-up cost of its parallel workers, not sampling.

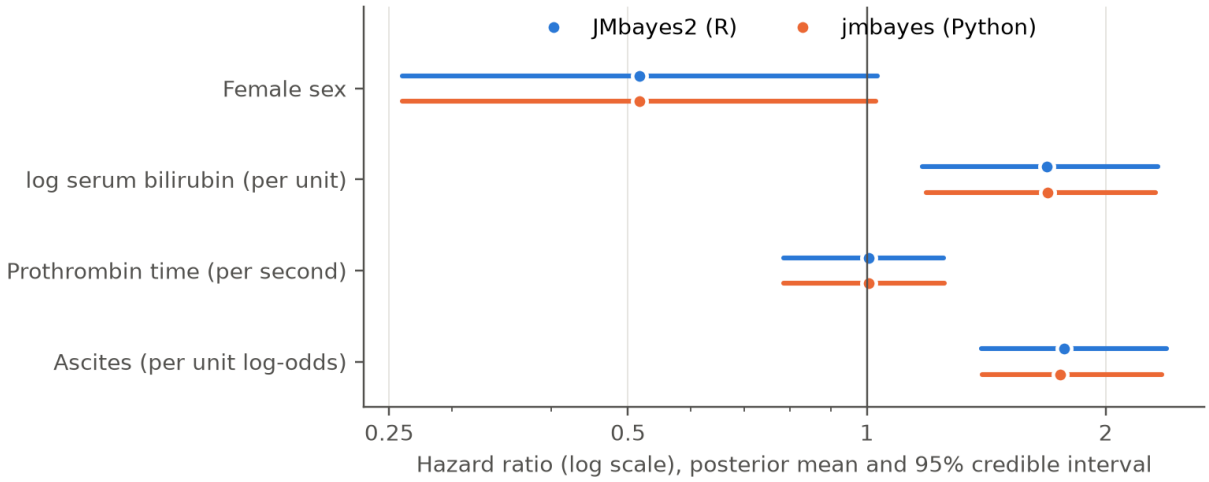
## Model and data

PBC data (pbc2, 312 patients, 1,945 visits, 169 deaths or transplants). The model is the one in the JMbayes2 README: a Cox-type survival model with sex, linked through the current value of three longitudinal markers - log serum bilirubin and prothrombin time (linear mixed models, random intercept and slope) and ascites (logistic mixed model, random intercept and slope). MCMC: 12 chains, 30,000 iterations per chain, 5,000 burn-in, no thinning (300,000 retained draws per program), identical in both programs. The R model was fitted with `jm()`; the Python model with `jmbayes`, using its own versions of the separate starting fits (lme, mixed_model, coxph).

## 1. Do the two implementations agree?

**Survival sub-model: hazard ratios.** Posterior mean hazard ratio with 95% credible interval (exp of the log-scale estimates) and the Bayesian tail probability P (2 x min(P(β>0), P(β<0)), as reported by JMbayes2).

| Effect on the hazard of death or transplant | R: HR (95% CI) | R: P | Python: HR (95% CI) | Python: P |
|---|---|---|---|---|
| Female sex | 0.52 (0.26 - 1.03) | 0.061 | 0.52 (0.26 - 1.03) | 0.060 |
| log serum bilirubin (per unit) | 1.68 (1.18 - 2.33) | 0.006 | 1.69 (1.19 - 2.31) | 0.006 |
| Prothrombin time (per second) | 1.01 (0.79 - 1.25) | 0.919 | 1.00 (0.79 - 1.25) | 0.933 |
| Ascites (per unit log-odds) | 1.77 (1.40 - 2.39) | <0.001 | 1.75 (1.40 - 2.35) | <0.001 |




### log(serum bilirubin) - linear mixed model

| Coefficient | R: estimate (95% CI) | R: P | Python: estimate (95% CI) | Python : P |
|---|---|---|---|---|
| (Intercept) | 0.699 (0.374, 1.024) | <0.001 | 0.696 (0.372, 1.022) | <0.001 |
| year | 0.277 (0.210, 0.347) | <0.001 | 0.278 (0.210, 0.346) | <0.001 |
| sexfemale | -0.241 (-0.586, 0.103) | 0.168 | -0.237 (-0.581, 0.106) | 0.176 |
| year:sexfemale | -0.084 (-0.155, -0.013) | 0.019 | -0.083 (-0.154, -0.013) | 0.021 |

### Prothrombin time - linear mixed model

| Coefficient | R: estimate (95% CI) | R: P | Python: estimate (95% CI) | Python : P |
|---|---|---|---|---|
| (Intercept) | 10.942 (10.607, 11.275) | <0.001 | 10.939 (10.605, 11.272) | <0.001 |
| year | 0.251 (0.106, 0.397) | <0.001 | 0.251 (0.106, 0.396) | <0.001 |
| sexfemale | -0.403 (-0.757, -0.049) | 0.026 | -0.401 (-0.754, -0.046) | 0.027 |
| year:sexfemale | 0.052 (-0.099, 0.204) | 0.496 | 0.053 (-0.098, 0.205) | 0.487 |

### Ascites - logistic mixed model

| Coefficient | R: estimate (95% CI) | R: P | Python: estimate (95% CI) | Python : P |
|---|---|---|---|---|
| (Intercept) | -4.613 (-6.135, -3.255) | <0.001 | -4.646 (-6.178, -3.271) | <0.001 |
| year | 0.652 (0.508, 0.811) | <0.001 | 0.663 (0.515, 0.830) | <0.001 |
| sexfemale | -0.493 (-1.861, 0.895) | 0.476 | -0.484 (-1.867, 0.911) | 0.486 |

| Residual standard deviation | R: mean (SD) | Python: mean (SD) |
|---|---|---|
| log(serum bilirubin) | 0.3468 (0.0067) | 0.3467 (0.0066) |
| Prothrombin time | 1.0547 (0.0203) | 1.0546 (0.0202) |

**Same conclusions.** 15 of 15 regression coefficients lead to the same conclusion in both programs.

| Coefficient | Same direction | Same significance (P < 0.05) | Same 'CI excludes 0' |
|---|---|---|---|
| Survival: sexfemale | yes | yes | yes |
| Survival: value(log(serBilir)) | yes | yes | yes |
| Survival: value(prothrombin) | yes | yes | yes |
| Survival: value(ascites) | yes | yes | yes |
| log(serum bilirubin): (Intercept) | yes | yes | yes |
| log(serum bilirubin): year | yes | yes | yes |
| log(serum bilirubin): sexfemale | yes | yes | yes |
| log(serum bilirubin): year:sexfemale | yes | yes | yes |
| Prothrombin time: (Intercept) | yes | yes | yes |
| Prothrombin time: year | yes | yes | yes |
| Prothrombin time: sexfemale | yes | yes | yes |
| Prothrombin time: year:sexfemale | yes | yes | yes |
| Ascites: (Intercept) | yes | yes | yes |
| Ascites: year | yes | yes | yes |
| Ascites: sexfemale | yes | yes | yes |

## Formal equivalence test (all 50 parameters)

**Step 1: realistic error bars.** Any comparison of two MCMC runs needs the Monte Carlo standard error (MCSE) of each estimate. JMbayes2 reports MCSE = SD / sqrt(effective sample size), with the effective sample size from an autoregressive fit to all chains stacked together. To check this formula we compared two runs of the same program that differ only in their random seed: their differences should be within ±2 MCSE about 95% of the time. For Python only 62% were, for R only 80%, so the formula understates the real run-to-run error by a factor k = 1.67 (k = sqrt(mean $z^2$) over all parameters and both same-program pairs). All error bars below are multiplied by k; the correction applies equally to R and Python.

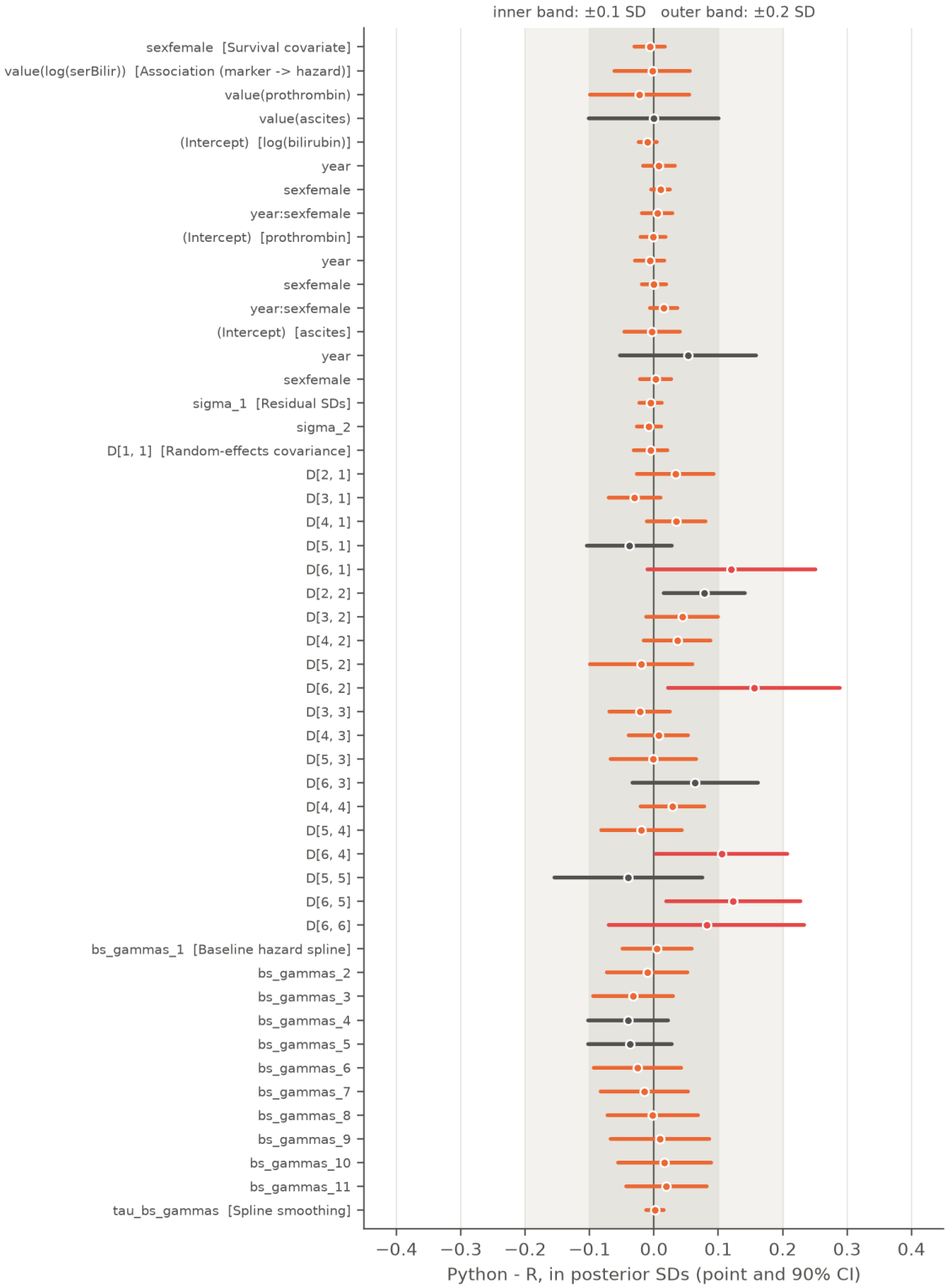
**Step 2: equivalence test.** A non-significant difference test only says a difference could not be detected. An **equivalence test (TOST)** asks whether the difference is small enough to be irrelevant. For each parameter, d = Python mean - R mean (each pooled over its seeds), with SE = k · sqrt($MCSE_R^2$ + $MCSE_{Py}^2$). The programs are declared equivalent for that parameter if the 90% confidence interval d ± 1.645·SE lies inside ±δ (two one-sided tests at $\alpha = 0.05$). The margin is relative to the parameter's own posterior uncertainty: $\delta = 0.1$ posterior SD (strict) or 0.2 SD. A shift of 0.2 SD moves a 95% credible interval by about 5% of its width.

**Benchmark: how different are two runs of the same program?**

| Comparison | Median \|difference\| | Largest \|difference\| |
|---|---|---|
| R vs Python (pooled seeds) | 0.018 SD | 0.155 SD |
| Python seed 123 vs Python seed 456 | 0.047 SD | 0.199 SD |
| R seed 123 vs R seed 456 | 0.023 SD | 0.141 SD |

| Parameter group | Parameters | Equivalent at ±0.1 SD | Equivalent at ±0.2 SD | Largest \|d\| (SD) |
|---|---|---|---|---|
| Survival covariate | 1 | 1 | 1 | 0.01 |
| Association (marker -> hazard) | 3 | 2 | 3 | 0.02 |
| Fixed effects: log(bilirubin) | 4 | 4 | 4 | 0.01 |
| Fixed effects: prothrombin | 4 | 4 | 4 | 0.02 |
| Fixed effects: ascites | 3 | 2 | 3 | 0.05 |
| Residual SDs | 2 | 2 | 2 | 0.01 |
| Random-effects covariance | 21 | 12 | 16 | 0.16 |
| Baseline hazard spline | 11 | 9 | 11 | 0.04 |
| Spline smoothing | 1 | 1 | 1 | 0.00 |
| **All** | **50** | **37** | **45** | **0.16** |

Figure: each parameter's difference (Python - R) in posterior SDs with its 90% CI (calibrated error bars). Orange: equivalent at ±0.1 SD; grey: equivalent at ±0.2 SD only; red: not shown equivalent at ±0.2 SD. Wide intervals belong to parameters that mix slowly in both programs (the random-effects covariances, mainly the ascites random slope, and the baseline-hazard spline); their estimates still sit close to zero difference, and longer runs would narrow the intervals.




**Noise check.** If both programs sample the same posterior, z = d / SE behaves like a standard normal. Observed: 98% of |z| below 2 (expected ~95%) and 100% below 3 (expected ~99.7%).

**Model fit statistics.** DIC, WAIC and LPML are themselves Monte Carlo estimates. The table puts the R-vs-Python gap next to the gap between two Python seeds; the gaps are of comparable size, so the fit statistics agree within run-to-run variation (relative differences are below 0.3%).

| Fit statistic (conditional) | R | Python (seed 123) | Python (seed 456) | \|R - Python mean\| | \|Python - Python\| |
|---|---|---|---|---|---|
| DIC | 12606.6 | 12631.9 | 12598.7 | 8.7 | 33.1 |
| pD | 1716.5 | 1719.9 | 1717.1 | 2.0 | 2.7 |
| WAIC | 12354.8 | 12375.9 | 12348.6 | 7.4 | 27.3 |
| LPML | -6823.5 | -6842.6 | -6831.9 | 13.8 | 10.7 |

## Convergence: Rhat in both programs

Rhat (Gelman-Rubin potential scale reduction) compares the spread within each chain to the spread between chains; values near 1 mean the chains agree, and JMbayes2's usual threshold is 1.05. Both programs compute it the same way (the point estimate of `coda::gelman.diag`, which JMbayes2 reports). The R output stores Rhat for the regression coefficients and residual SDs only, so those are compared here; across all 50 parameters, the Python runs reach at most 1.088.

**Long runs** (12 chains, 30,000 iterations per chain, 5,000 burn-in, no thinning (300,000 retained draws per program))

| Parameter | R, seed 123 | R, seed 456 | Python, seed 123 | Python, seed 456 |
|---|---|---|---|---|
| Survival: sexfemale | 1.002 | 1.003 | 1.001 | 1.001 |
| Survival: value(log(serBilir)) | 1.008 | 1.010 | 1.007 | 1.008 |
| Survival: value(prothrombin) | 1.011 | 1.029 | 1.020 | 1.013 |
| Survival: value(ascites) | 1.023 | 1.032 | 1.019 | 1.017 |
| log(bilirubin): (Intercept) | 1.001 | 1.000 | 1.000 | 1.000 |
| log(bilirubin): year | 1.002 | 1.003 | 1.001 | 1.003 |
| log(bilirubin): sexfemale | 1.001 | 1.000 | 1.000 | 1.000 |
| log(bilirubin): year:sexfemale | 1.002 | 1.002 | 1.002 | 1.003 |
| log(bilirubin): residual SD | 1.001 | 1.000 | 1.001 | 1.001 |
| prothrombin: (Intercept) | 1.002 | 1.001 | 1.001 | 1.001 |
| prothrombin: year | 1.001 | 1.001 | 1.001 | 1.002 |
| prothrombin: sexfemale | 1.002 | 1.001 | 1.001 | 1.001 |
| prothrombin: year:sexfemale | 1.002 | 1.001 | 1.001 | 1.002 |
| prothrombin: residual SD | 1.001 | 1.002 | 1.000 | 1.001 |
| ascites: (Intercept) | 1.012 | 1.020 | 1.010 | 1.006 |
| ascites: year | 1.024 | 1.043 | 1.041 | 1.015 |
| ascites: sexfemale | 1.002 | 1.002 | 1.001 | 1.001 |
| **Largest** | **1.024** | **1.043** | **1.041** | **1.017** |

The pattern is the same in both programs: the longitudinal coefficients are at 1.00, and the slowest-mixing parameters are the same ones - the ascites association and the ascites (logistic) sub-model. All stay below 1.05 in every run.

**Short runs** (scaling runs in Section 2: 3 chains, burn-in 500, thinning 5). Each cell: largest Rhat, number of the 15 coefficients above 1.05, and the worst coefficient.

| Patients | Iterations | JMbayes2 | Python, GPU | Python, CPU |
|---|---|---|---|---|
| 312 | 3,500 | 1.78, 3/15 (ascites: year) | 1.19, 4/15 (Survival: value(prothrombin)) | 1.45, 5/15 (Survival: value(ascites)) |
| 624 | 3,500 | 1.36, 4/15 (Survival: value(prothrombin)) | 1.48, 5/15 (ascites: year) | 1.68, 4/15 (Survival: value(prothrombin)) |
| 1,560 | 3,500 | 1.44, 5/15 (ascites: year) | 1.32, 5/15 (ascites: year) | 1.63, 6/15 (ascites: year) |
| 3,120 | 3,500 | 1.58, 6/15 (Survival: value(log(serBilir))) | 1.77, 5/15 (Survival: value(ascites)) | 1.33, 5/15 (Survival: value(prothrombin)) |
| 1,560 | 7,000 | 1.60, 3/15 (ascites: year) | 1.11, 3/15 (ascites: year) | 2.44, 5/15 (ascites: year) |
| 3,120 | 7,000 | 1.54, 4/15 (Survival: value(prothrombin)) | 1.90, 4/15 (ascites: year) | 1.23, 3/15 (ascites: year) |
| 1,560 | 14,000 | 1.42, 4/15 (ascites: year) | 1.13, 1/15 (ascites: year) | - |
| 3,120 | 14,000 | 1.18, 4/15 (Survival: value(prothrombin)) | 1.36, 4/15 (ascites: year) | - |
| 1,560 | 28,000 | 1.13, 2/15 (ascites: year) | 1.18, 2/15 (ascites: year) | - |
| 3,120 | 28,000 | 1.01, 0/15 (Survival: value(log(serBilir))) | 1.05, 1/15 (Survival: value(prothrombin)) | - |
| 1,872 | 30,000 | 1.06, 1/15 (Survival: value(prothrombin)) | - | - |
| 1,560 | 60,000 | - | 1.01, 0/15 (ascites: year) | - |
| 3,120 | 60,000 | 1.03, 0/15 (ascites: year) | 1.14, 2/15 (ascites: year) | - |
| 1,560 | 90,000 | - | 1.03, 0/15 (Survival: value(prothrombin)) | - |
| 3,120 | 90,000 | - | 1.05, 0/15 (ascites: year) | - |

With JMbayes2's default length (3,500 iterations) and 3 chains, neither program converges for the ascites-related parameters on this model: several coefficients are above 1.05 in R and in Python alike, at every cohort size. Rhat is itself noisy with only 3 chains, so the exact values move from run to run, but neither program is consistently better. For this model, real analyses need longer runs or more chains in either program - which is where the speed of jmbayes on a GPU matters most, since extra chains cost almost nothing there.

## Isolating the sampler: Python started from R's own starting fits

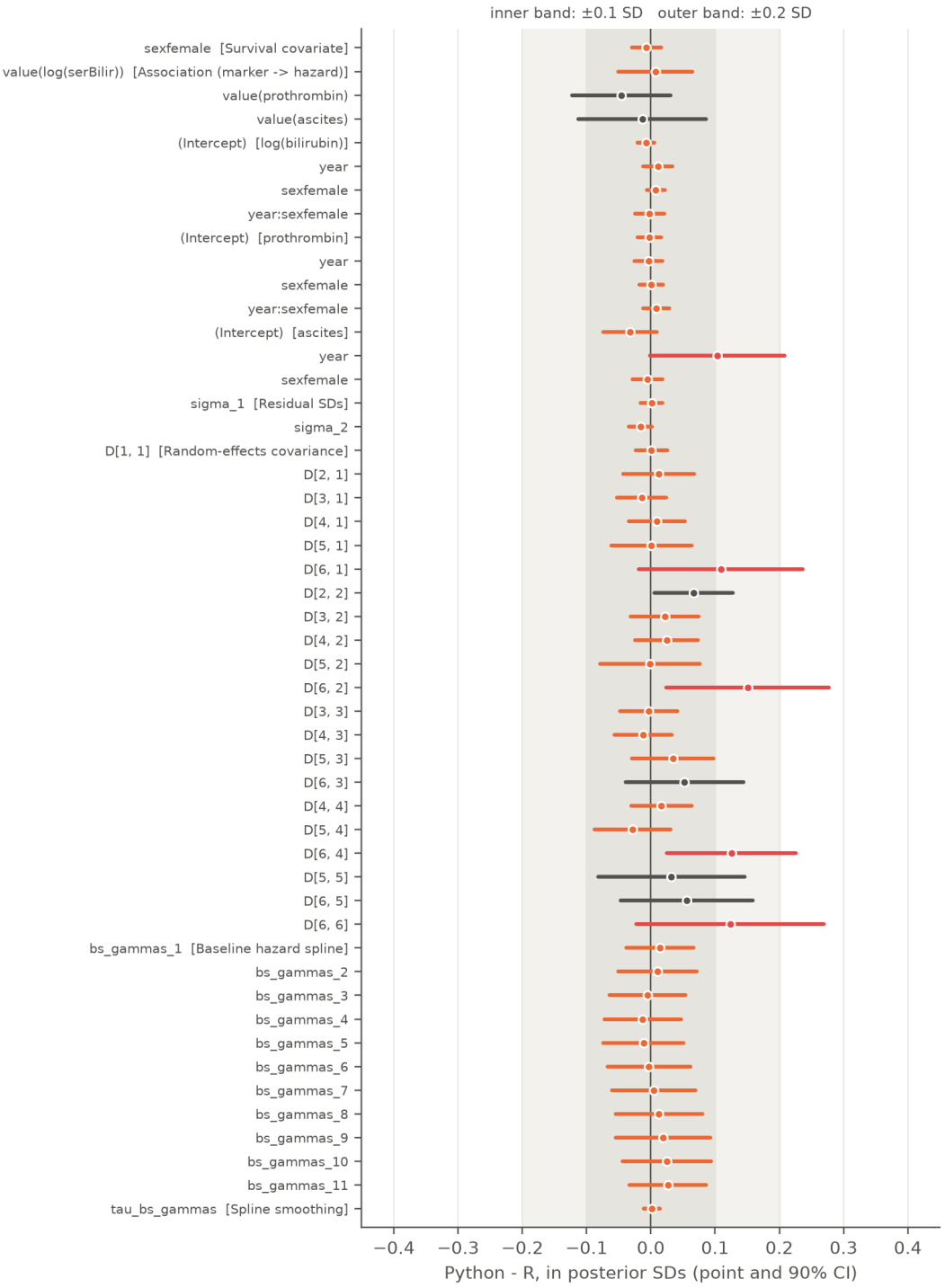
JMbayes2 does not start from scratch: it takes fitted lme / mixed_model / coxph objects, uses them as starting values and centres some priors on them (for example the prior for each random-effect standard deviation). The Python runs above used Python's own versions of those fits, which match R's to 4-5 digits for the linear mixed models but differ by about 1% for the ascites logistic mixed model. To remove that difference entirely, the R fits were exported (benchmark/export_initfits.R) and Python was run again from exactly those values, with the same MCMC settings and two seeds. Any remaining difference can then only come from the sampler itself.

| | Python, own starting fits | Python, R's starting fits |
|---|---|---|
| Median \|difference\| from R | 0.018 SD | 0.012 SD |
| Largest \|difference\| from R | 0.155 SD | 0.151 SD |
| Mean difference, ascites random-slope covariances D[6,·] | +0.108 SD | +0.103 SD |
| Equivalent at ±0.1 SD (TOST) | 37 / 50 | 39 / 50 |
| Equivalent at ±0.2 SD (TOST) | 45 / 50 | 45 / 50 |
| \|z\| below 2 (expected ~95%) | 98% | 98% |
| \|z\| below 3 (expected ~99.7%) | 100% | 100% |
| Parameters shown to differ (90% CI outside ±0.2 SD) | 0 | 0 |

**The parameters that differed most** (posterior means; every column averages two seeds)

| Parameter | R | Python (own fits) | Python (R's fits) |
|---|---|---|---|
| ascites: year | 0.6530 | 0.6571 | 0.6611 |
| D[6, 1] | 0.2075 | 0.2151 | 0.2144 |
| D[6, 2] | 0.0634 | 0.0658 | 0.0657 |
| D[6, 3] | 0.0892 | 0.0924 | 0.0918 |
| D[6, 4] | 0.0804 | 0.0830 | 0.0835 |
| D[6, 5] | -0.0321 | -0.0060 | -0.0203 |
| D[6, 6] | 0.2448 | 0.2513 | 0.2546 |

Figure: as before, R vs Python started from R's fits (calibration factor k = 1.60, estimated from the same-program pairs).

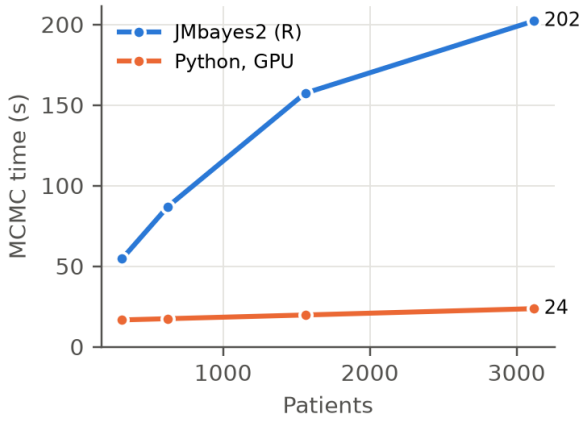

inner band: ±0.1 SD  outer band: ±0.2 SD
sexfemale [Survival covariate]
value(log(serBilir)) [Association (marker -> hazard)]
value(prothrombin)
value(ascites)
(Intercept) [log(bilirubin)]
year
sexfemale
year:sexfemale
(Intercept) [prothrombin]
year
sexfemale
year:sexfemale
(Intercept) [ascites]
year
sexfemale
sigma_1 [Residual SDs]
sigma_2
D[1, 1] [Random-effects covariance]
D[2, 1]
D[3, 1]
D[4, 1]
D[5, 1]
D[6, 1]
D[2, 2]
D[3, 2]
D[4, 2]
D[5, 2]
D[6, 2]
D[3, 3]
D[4, 3]
D[5, 3]
D[6, 3]
D[4, 4]
D[5, 4]
D[6, 4]
D[5, 5]
D[6, 5]
D[6, 6]
bs_gammas_1 [Baseline hazard spline]
bs_gammas_2
bs_gammas_3
bs_gammas_4
bs_gammas_5
bs_gammas_6
bs_gammas_7
bs_gammas_8
bs_gammas_9
bs_gammas_10
bs_gammas_11
tau_bs_gammas [Spline smoothing]
−0.4
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
0.4
Python - R, in posterior SDs (point and 90% CI)

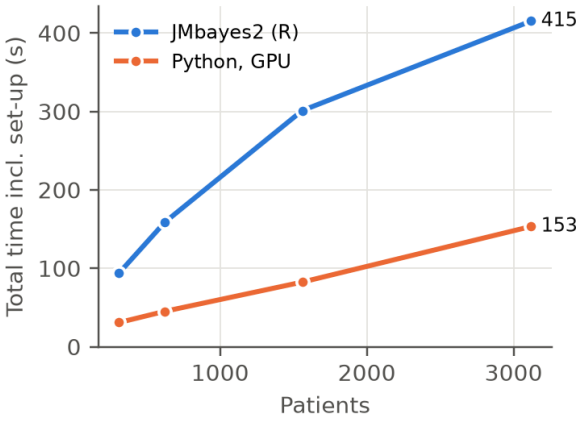
## 2. Speed and memory (preliminary)

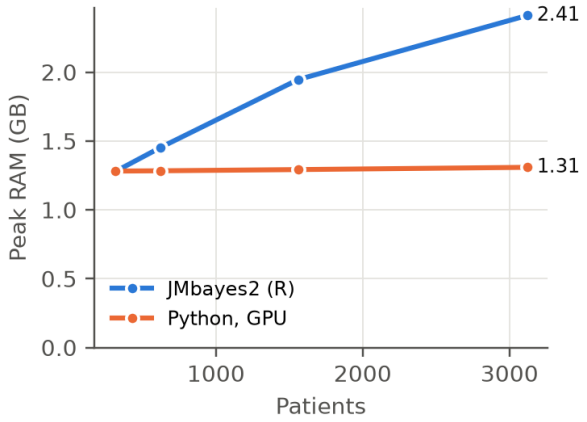
*Preliminary: the sweep was paused to run the equivalence test. The Python run at 28,000 iterations and the colleague-scale runs (30,000-90,000 iterations) will be added in a later version. The R run at 28,000 iterations overlapped with another R run on this 24-core machine, so its time may be slightly inflated.*

Each run is a separate process, run one at a time. Peak RAM is the resident memory of the whole process tree, sampled every 0.2 s (R starts one worker process per chain; all are counted). Larger cohorts are pbc2 copied 2, 5 and 10 times with new patient ids. 3 chains, burn-in 500, thinning 5. **MCMC time** is the sampling step; for R it includes starting its parallel worker processes. **Total time** also includes the separate starting fits (lme, mixed_model, coxph) that both programs need.

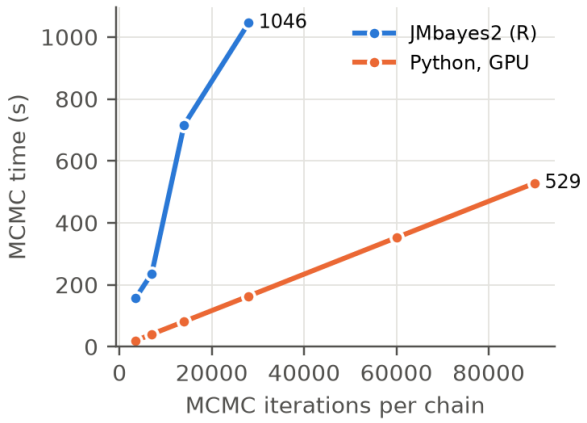
**Scaling with the number of patients** (3,500 iterations, JMbayes2's default)

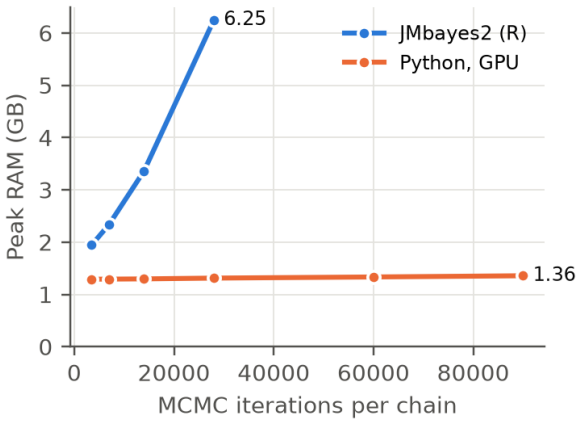



**Scaling with the number of iterations** (1,560 patients)



**All measurements**

| Implementation | Patients | Iterations | MCMC (s) | Total (s) | Peak RAM (GB) | GPU memory (GB) |
|---|---|---|---|---|---|---|
| Python, CPU | 312 | 3,500 | 66.8 | 80.4 | 0.56 | - |
| Python, GPU | 312 | 3,500 | 16.7 | 30.9 | 1.28 | 0.03 |
| JMbayes2 (R) | 312 | 3,500 | 54.6 | 94.1 | 1.28 | - |
| Python, CPU | 624 | 3,500 | 83.6 | 110.7 | 0.57 | - |
| Python, GPU | 624 | 3,500 | 17.4 | 45.2 | 1.28 | 0.04 |
| JMbayes2 (R) | 624 | 3,500 | 87.0 | 158.9 | 1.45 | - |
| Python, CPU | 1,560 | 3,500 | 140.0 | 201.9 | 0.61 | - |
| Python, GPU | 1,560 | 3,500 | 19.7 | 82.5 | 1.29 | 0.05 |
| JMbayes2 (R) | 1,560 | 3,500 | 157.4 | 301.2 | 1.95 | - |
| Python, CPU | 3,120 | 3,500 | 445.2 | 774.7 | 0.67 | - |
| Python, GPU | 3,120 | 3,500 | 23.6 | 153.4 | 1.31 | 0.07 |
| JMbayes2 (R) | 3,120 | 3,500 | 202.2 | 415.4 | 2.41 | - |
| Python, CPU | 1,560 | 7,000 | 568.2 | 723.1 | 0.60 | - |
| Python, GPU | 1,560 | 7,000 | 40.7 | 196.8 | 1.29 | 0.05 |
| JMbayes2 (R) | 1,560 | 7,000 | 236.8 | 335.0 | 2.34 | - |
| Python, CPU | 3,120 | 7,000 | 476.5 | 605.9 | 0.67 | - |
| Python, GPU | 3,120 | 7,000 | 47.9 | 177.1 | 1.31 | 0.07 |
| JMbayes2 (R) | 3,120 | 7,000 | 450.4 | 758.3 | 3.44 | - |
| Python, GPU | 1,560 | 14,000 | 81.6 | 238.1 | 1.30 | 0.06 |
| JMbayes2 (R) | 1,560 | 14,000 | 716.4 | 919.4 | 3.36 | - |
| Python, GPU | 3,120 | 14,000 | 96.4 | 226.5 | 1.32 | 0.08 |
| JMbayes2 (R) | 3,120 | 14,000 | 521.2 | 698.7 | 6.15 | - |
| Python, GPU | 1,560 | 28,000 | 163.7 | 320.8 | 1.31 | 0.08 |
| JMbayes2 (R) | 1,560 | 28,000 | 1045.8 | 1333.4 | 6.25 | - |
| Python, GPU | 3,120 | 28,000 | 195.7 | 326.4 | 1.33 | 0.10 |
| JMbayes2 (R) | 3,120 | 28,000 | 2055.8 | 2439.5 | 12.02 | - |
| JMbayes2 (R) | 1,872 | 30,000 | 1272.6 | 1519.4 | 7.94 | - |
| Python, GPU | 1,560 | 60,000 | 352.4 | 509.1 | 1.33 | 0.12 |
| Python, GPU | 3,120 | 60,000 | 428.6 | 760.3 | 1.39 | 0.14 |
| JMbayes2 (R) | 3,120 | 60,000 | 5893.4 | 6733.3 | 25.24 | - |
| Python, GPU | 1,560 | 90,000 | 528.6 | 684.9 | 1.36 | 0.16 |
| Python, GPU | 3,120 | 90,000 | 642.9 | 972.7 | 1.38 | 0.18 |

## 3. What was ported, and caveats

• **Same algorithm.** Python follows JMbayes2's C++ sampler step for step: the same parameter blocks in the same order, the same proposal types, the same Gibbs steps, the same adaptive step sizes and burn-in rules. Only the random-number stream differs, so individual draws differ while the results agree within Monte Carlo error.

• **How it is faster.** All chains and all patients are updated together as tensor operations, and on the GPU each iteration is recorded once and replayed (CUDA graph). The per-patient log-likelihood is not stored for every iteration; DIC, WAIC and LPML are accumulated during sampling, which gives identical values (checked to 1e-16).

• **Starting fits.** JMbayes2 takes fitted lme / mixed_model / coxph objects, which also set some prior centres. Python re-implements these estimators (REML, ML with adaptive Gauss-Hermite quadrature, Cox with Efron ties); they match to optimiser tolerance.

• **Two possible issues in JMbayes2's C++** were found and deliberately reproduced so results match: the random-effects log-density is not refreshed after the fixed-effects step, and the survival denominator is not refreshed after the smoothing-parameter update. Both are worth raising with the package authors.

• **Timing caveats.** A first R run on pbc2 took 143 s of MCMC and a repeat 58.5 s, so single timings vary. On large copies, R's own nlme starting fit failed to converge with its default optimiser; the optim() fallback was used only in that case.

• **Scope.** The Python version currently covers what this model needs: Gaussian and binomial outcomes, current-value association, right censoring. Other JMbayes2 features (more families, slope/area associations, competing risks, recurrent events) are not ported yet, and marginal fit statistics are not computed.